

Ants

Day 3 solution walkthrough

Competitive Programming Bootcamp

Statement excerpts:

- ▶ *“earliest and the latest possible times”*
- ▶ *“we do not know the directions”*

Programming interpretation:

- ▶ Colliding ants can be treated as passing through each other because the ants are indistinguishable.
- ▶ Earliest time uses each ant's nearest end; latest time uses each ant's farthest end.
- ▶ The answer is the maximum of those per-ant times.

Textbook connection: CP4 3.4 e. Non-Classical Greedy, Easier

Core technique: Collision equivalence observation

- ▶ Ants walk on a pole and turn around when they collide.
- ▶ We need earliest and latest possible time for all ants to fall.
- ▶ Ant identities do not matter for the final fall times.

- ▶ When two identical ants collide and swap directions, it is equivalent to them passing through each other.
- ▶ Therefore every ant independently chooses an end to fall from.
- ▶ Earliest uses the closest end; latest uses the farthest end.

- ▶ For each ant position p , compute left distance p .
- ▶ Compute right distance $\text{length} - p$.
- ▶ Update earliest with $\max(\min(\text{left}, \text{right}))$.
- ▶ Update latest with $\max(\max(\text{left}, \text{right}))$.
- ▶ Print earliest and latest for each test case.

- ▶ The last ant to fall determines the total time.
- ▶ For earliest time, make every ant fall by its nearer endpoint.
- ▶ For latest time, consider the farthest endpoint for each ant.

- ▶ The pass-through model preserves the multiset of positions over time.
- ▶ Earliest and latest fall times depend only on distances to endpoints, not names of ants.
- ▶ Taking the maximum over individual fall distances gives the time when all ants have fallen.

Complexity: $O(N)$ time per test case and $O(1)$ extra memory apart from positions.

- ▶ No pairwise collision simulation is needed.
- ▶ Positions can span multiple input lines, so read until all positions are collected.

Source lines: ants.py:36-52

```
import sys

def main():
    input = sys.stdin.buffer.readline

    cases = int(input())

    for _ in range(cases):
        length, n = map(int, input().split())

        positions = []

        while len(positions) < n:
            positions.extend(map(int, input().split()))
```

What this chunk does

- ▶ Each test case gives the pole length and number of ants.
- ▶ Positions may span several lines, so the loop keeps reading until n positions are collected.
- ▶ The solution does not need to store directions.

Source lines: ants.py:53-71

```
earliest = 0
latest = 0

for position in positions:
    left = position
    right = length - position

    closest_end = min(left, right)
    earliest = max(earliest, closest_end)

    farthest_end = max(left, right)
    latest = max(latest, farthest_end)

print(earliest, latest)

main()
```

What this chunk does

- ▶ For one ant, left and right are the two possible falling times.
- ▶ The earliest global time is the largest nearest-end distance.
- ▶ The latest global time is the largest farthest-end distance.

Solution file: `practice_contests/day_3/ants.py`

Problem: <https://open.kattis.com/problems/ants>

Use this deck with the source code open beside it. The slides explain the reasoning; the code shows the exact input handling and output formatting.